EMERGENCY-ROOM TRIAGE PRIORITY QUEUE

Using a Max-Heap — Medicine and Public Health (SDG 3)

ABSTRACT:

The Emergency-Room Triage Priority Queue is a data-structure-driven application designed to make sure that the most urgent patient is always treated next, while still respecting the order in which patients arrived. In a real emergency department, patients do not arrive in order of seriousness. A simple first-come-first-served queue would make a critically injured person wait behind a patient with a minor complaint, and a plain sorted list is too slow to re-arrange every time a new patient walks in.

This project solves the problem with a **Max-Heap**. Each node represents one patient and stores the Patient Name, Severity (1 to 5, where 5 is the most critical) and Arrival Time. The heap is ordered by severity first and, when two patients have the same severity, by earlier arrival time. This makes the root of the heap always the next patient to be treated, which can be viewed in O(1) time and removed in O(log n) time.

Core operations implemented include: Insert patient — for registering a new arrival; Peek — for viewing the next patient without removing them; Extract-Max — for treating the highest-priority patient; Update Severity — for re-prioritising a patient whose condition has changed; and Display — for listing all waiting patients in treatment order. The system also includes a simulation of arrivals and treatments that reports the treatment order and the waiting time of every patient, tests for ties and an empty queue, an analysis of the O(log n) cost, a comparison with linear search, and a discussion of triage fairness. It contributes to **SDG 3 (Good Health and Well-Being)** by supporting faster and more consistent emergency care.

Keywords: Max-Heap, Priority Queue, Triage, Emergency Room, Severity, Arrival Time, Heapify, Data Structures, SDG 3.

INTRODUCTION:

In an emergency room, time directly affects survival. Manual triage and simple waiting lines are slow, error-prone and sometimes unfair to patients whose condition is the most serious. A well-designed triage system must answer one question quickly and correctly: *who should be treated next?*

This project implements an Emergency-Room Triage Priority Queue using a Max-Heap data structure, where each node stores the patient's name, severity and arrival time. The heap keeps the most urgent patient at the root at all times, so the question above can be answered immediately, and the structure re-organises itself in logarithmic time after every insertion, treatment or change of severity.

PROBLEM STATEMENT:

Always treat the most urgent patient next while respecting arrival time. When two patients have the same severity, the one who arrived earlier must be treated first.

REQUIREMENTS COVERED:

- Max-heap keyed by severity and arrival time
- Operations: insert, extract-max, update severity and peek
- Simulation of arrivals and treatments, with the treatment order and waiting times as output
- Testing of ties (equal severity) and of the empty queue
- Analysis of the O(log n) cost of heap operations
- Comparison with a linear-search priority list
- Discussion of triage fairness

ALGORITHM:

Step 1: Start
Step 2: Define Node Structure
Each node represents one patient:
    Patient:
    name
    severity (1 to 5, 5 = most critical)
    arrivalTime (increasing counter)
Step 3: Initialize
    Create an empty heap array: heap[100], size = 0
    arrival = 0
Step 4: Define priority rule higherPriority(a, b)
    If a.severity != b.severity: return a.severity > b.severity
    Else: return a.arrivalTime < b.arrivalTime (earlier arrival wins ties)
Step 5: Display Menu (Repeat until exit)
    Add Patient / View Next Patient / Treat Patient / Update Severity / Display Patients / Exit
Step 6: Perform Operations
    6a. Add Patient
        Input name and severity; if severity not in 1..5 print "Invalid severity" and stop
        Set arrivalTime = ++arrival
        If size == 100 print "Queue is full" and stop
        Place patient at heap[size], size++
        Sift up: while i > 0 and higherPriority(heap[i], heap[parent]) swap with parent
    6b. View Next Patient (Peek)
        If size == 0 print "No patients available"; else print heap[0]
    6c. Treat Patient (Extract-Max)
        If size == 0 print "No patients to treat" and stop
        Save heap[0]; move heap[size-1] to heap[0]; size--
        Heapify(0): swap with the higher-priority child until the heap property is restored
        Print the treated patient
    6d. Update Severity
        Input name and new severity (1..5)
        Search the heap for the name; if found set the new severity and rebuild the heap
        If not found print "Patient not found"
    6e. Display Patients
        If size == 0 print "No patients available"
        Else copy the heap and repeatedly extract the maximum from the copy, printing each patient
    6f. Exit — stop program
Step 7: Stop

FLOW CHART:

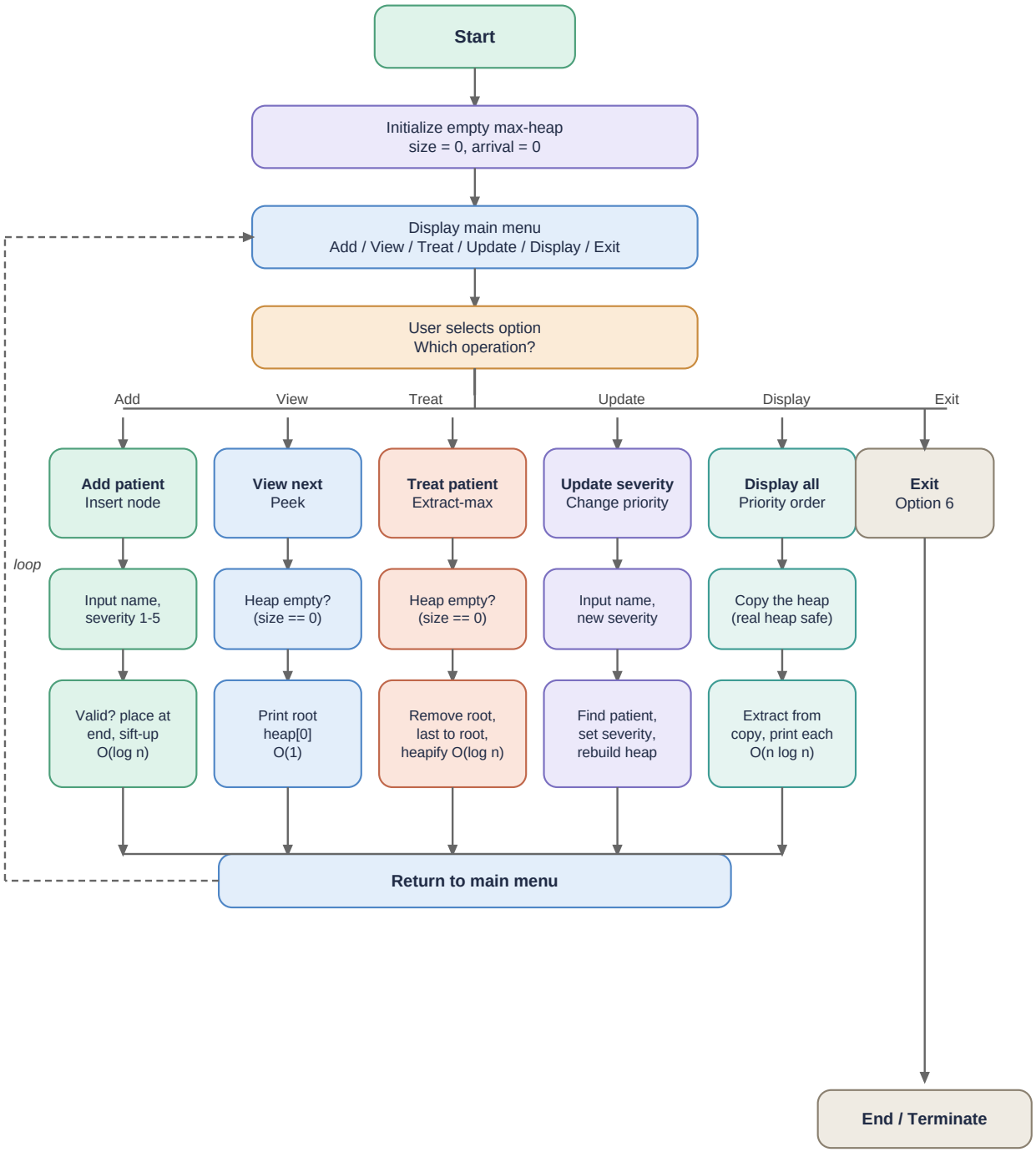

Fig. 1: Flow chart of the menu-driven triage system.

METHODOLOGY:

The triage system is built around a **binary Max-Heap stored in an array**. For the element at index i, the parent is at (i - 1) / 2, the left child at 2i + 1 and the right child at 2i + 2. Because the heap is a complete binary tree, the array is always packed without gaps and no pointers are needed.

Priority key. Every patient carries two values. The comparison function `higherPriority(a, b)` first compares severity (larger is more urgent). If the severities are equal, it compares arrival time and the patient who arrived *earlier* wins. Arrival time is a counter that increases with each registered patient, so no two patients ever have the same key and the order is always well defined.

Insert (sift-up). The new patient is placed in the first free position and is swapped with its parent while it has higher priority than the parent. At most one swap per tree level is needed. **Extract-Max (heapify-down).** The root is treated, the last patient is moved to the root and then pushed down by swapping with the higher-priority child until the heap property holds again. **Peek** simply reads the root.

Update severity. The patient is located by name, the severity is changed and the heap is rebuilt by calling heapify on every internal node from size/2 - 1 down to 0. This handles both a patient whose condition worsens (must move up) and one who improves (must move down).

Display. To list patients in treatment order without disturbing the live queue, the heap is copied and patients are extracted from the copy one at a time.

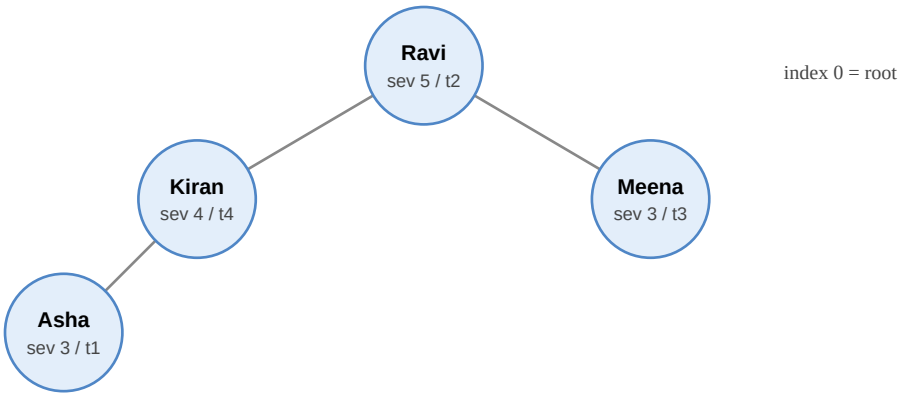

Fig. 2: Heap after inserting Asha(3), Ravi(5), Meena(3), Kiran(4). Array form: [Ravi, Kiran, Meena, Asha]. Array position is not treatment order: the order is Ravi, Kiran, Asha, Meena, because Asha arrived before Meena at the same severity.

Simulation method. A single doctor treats one patient at a time. Whenever the doctor becomes free, all arrivals (and severity updates) that happened up to that moment are applied to the heap, then Extract-Max selects the next patient. The waiting time of a patient is *treatment start time – arrival time*.

COMPLEXITY ANALYSIS (O(log n)):

A heap with n patients is a complete binary tree of height floor($\log_2$ n). Sift-up and heapify-down each move along a single root-to-leaf path and do a constant amount of work per level, so their cost is proportional to the height, i.e. O(log n).

Operation	Time	Why
Peek	O(1)	Reads heap[0] only
Insert	O(log n)	At most log n swaps going up one path
Extract-Max	O(log n)	Heapify goes down one path, 2 comparisons per level
Update severity (as implemented)	O(n)	O(n) name search + O(n) bottom-up rebuild
Update severity (with index map)	O(log n)	Find by index in O(1), then sift-up or sift-down
Display all (copy + extract all)	O(n log n)	n extractions of O(log n) each
Space	O(n)	Fixed array of 100 patients (plus one temporary copy in Display)

Total cost of treating n patients: n insertions plus n extractions give O(n log n) in total, compared with O(n^2) for a linear list (see next section). For a hospital queue of n = 100, a heap needs at most about 7 swaps per operation ($\log_2 100 \approx 6.6$).

COMPARISON WITH LINEAR SEARCH:

A simple alternative is an unsorted array where the next patient is found by scanning every element (linear search). Insert is O(1), but finding and removing the most urgent patient costs O(n). Both approaches were implemented with the same priority rule and run on random patients (severity 1 to 5) for several sizes, inserting n patients and then treating all n. The comparison column counts calls of `higherPriority`.

n patients	Heap comparisons	Linear comparisons	Heap time	Linear time
1,000	16,810	499,500	0.2 ms	2.5 ms
5,000	107,232	12,497,500	1.2 ms	63.6 ms
20,000	508,946	199,990,000	4.9 ms	1,172.9 ms
50,000	1,405,348	1,249,975,000	20.9 ms	7,927.1 ms

Timings were measured on the development machine with optimisation enabled; absolute values vary by machine, but the comparison counts match the theory exactly: the linear list needs n(n-1)/2 comparisons, while the heap grows roughly as n log n. At 50,000 patients the heap used about 900 times fewer comparisons. For very small queues (a handful of patients) the two are equally fast, but a heap keeps its guaranteed O(log n) behaviour when an emergency room becomes crowded — exactly the situation in which a slow decision is most harmful.

RESULT:

1. Console run of the program

```
Enter your choice: 1
Enter patient name: Asha
Enter severity (1-5): 3
Patient added successfully!
   ... (Ravi 5, Meena 3, Kiran 4 added the same way) ...
Enter your choice: 2
Next patient: Ravi
Severity: 5
Enter your choice: 5
Patients in treatment priority order:
Ravi - Severity: 5
Kiran - Severity: 4
Asha - Severity: 3
Meena - Severity: 3
Enter your choice: 4
Enter patient name: Meena
Enter new severity (1-5): 5
Severity updated successfully!
Enter your choice: 5
Patients in treatment priority order:
Ravi - Severity: 5
Meena - Severity: 5
Kiran - Severity: 4
Asha - Severity: 3
Enter your choice: 3
Patient treated: Ravi
Enter your choice: 3
Patient treated: Meena
Enter your choice: 6
Thank you!
```

Fig. 3: Sample console session (menu redisplay omitted for brevity). Meena was raised from severity 3 to 5 and moved ahead of Kiran, but behind Ravi, who arrived earlier at the same severity.

2. Simulation of arrivals and treatments

Eight patients arrive over 9 minutes and are treated by one doctor. At minute 6 Priya's condition worsens from severity 2 to 5. Times are in minutes.

Order	Patient	Severity at treatment	Arrival	Start	End	Waiting time
1	Asha	3	0	0	4	0
2	Ravi	5	1	4	9	3
3	Priya	5 (was 2)	5	9	11	4
4	Sam	5	8	11	15	3
5	Kiran	4	3	15	19	12
6	Meena	3	2	19	22	17
7	John	3	4	22	25	18
8	Lata	1	9	25	27	16

Average waiting time = 9.12 minutes; maximum waiting time = 18 minutes (John, severity 3). Asha was treated first only because she was alone when she arrived. After that, every treatment went to the highest severity present: Ravi (5) before Meena and Kiran, Priya after her update (5, arrived at minute 5) before Sam (5, arrived at minute 8), then Kiran (4), and the severity-3 patients in arrival order.

3. Test cases (ties and empty queue)

Test	Input	Expected	Observed
Empty queue – treat	Option 3 on empty heap	No patients to treat.	No patients to treat. (repeated safely after all patients were treated)
Empty queue – peek	Option 2 on empty heap	No patients available.	No patients available.
Empty queue – display	Option 5 on empty heap	No patients available.	No patients available.
Tie in severity	4 patients, all severity 3, arrivals t1..t4	Treated in arrival order	T1, T2, T3, T4
Tie after update	Meena raised 3 → 5 while Ravi is 5	Ravi (earlier) before Meena	Ravi, Meena, Kiran, Asha
Invalid severity	Severity 9 when adding	Rejected	Invalid severity! Enter 1 to 5.
Unknown name	Update severity of "Zed"	Not found message	Patient not found.
Single patient	Insert then extract	Heap becomes empty, no crash	Patient treated; next treat reports empty
Display keeps queue intact	Display, then treat repeatedly	Same order as shown	Same order as shown
Full queue	101st patient	Rejected	Queue is full!

All tests behaved as expected. One implementation note: the *Display* function works on a **copy** of the heap. Extracting from the live heap while printing would destroy the queue, so the copy guarantees that viewing the list never changes who is treated next.

TRIAGE FAIRNESS:

A priority queue is fair in a medical sense because it treats patients according to need, not according to who arrived first. The ordering rule here is clear and applies to everybody in the same way: higher severity first, and earlier arrival as the tie-breaker. Two patients with the same severity can never overtake each other, which gives predictable, explainable behaviour.

There are limits. A strict max-heap can cause **starvation**: in a busy emergency room, a patient with a low severity may wait a very long time while more serious patients keep arriving. In the simulation Lata (severity 1) waited 16 minutes and John (severity 3) 18 minutes, the longest of anyone. This is acceptable in real triage because the delay is paid by those who can safely wait, but it must be controlled. Possible remedies are **aging** (raising a patient's severity by one level after a fixed waiting time), a maximum-wait threshold that forces treatment, and the *Update Severity* operation, which lets nurses re-assess waiting patients so that a patient who deteriorates immediately moves forward.

The result of a triage system is only as fair as its inputs: severity scores must be assigned consistently and without bias (for example by a standard scale such as the five-level Emergency Severity Index). The data structure guarantees correct ordering for the given scores; it cannot correct a wrongly assigned score. For this reason the system should support the clinician, not replace clinical judgement.

CONCLUSION:

This project successfully demonstrates the design and implementation of an Emergency-Room Triage Priority Queue using a Max-Heap. The system always selects the most urgent patient next, and uses arrival time to break ties, so that equally urgent patients are treated in the order in which they came.

Through this project, the practical use of a heap in a life-critical, real-world situation has been shown. Peek takes constant time, while insertion and extraction take O(log n); the comparison with linear search showed a gap that grows quickly with the number of patients (about 900 times fewer comparisons at 50,000 patients). The simulation produced the treatment order and waiting times for a group of patients, including the effect of a patient whose condition worsened, and the tests on ties and the empty queue showed correct and safe behaviour.

From an academic perspective, this project strengthened the understanding of array-based trees, sift-up and heapify, custom comparison rules and algorithm analysis. It bridges the gap between the theory of priority queues and a practical application that supports **SDG 3: Good Health and Well-Being** by helping the people who need care most to receive it first.

FUTURE WORK:

Although the current system fulfils its basic objectives, several enhancements can be made in the future:

- **Faster Update Severity** — Keep a name/ID-to-index map so that updates run in O(log n) instead of O(n).
- **Built-in waiting-time reporting** — Record treatment start times inside the program and print waiting times for every patient automatically.
- **Aging to prevent starvation** — Gradually raise the priority of patients who have waited too long.
- **Dynamic capacity** — Replace the fixed array of 100 with a resizable array (vector).
- **Unique patient IDs** — Handle names with spaces and duplicate names safely.
- **File handling / database** — Save and load the queue so data survives program restarts.
- **Multiple doctors and departments** — Extend the simulation to several treatment rooms.
- **Graphical interface** — A dashboard for nurses showing the live queue and expected waiting times.